

Python practicals

1. Implement a program based on strings.(slicing, casting, 5 string function)

```
# String input
text = "Hello Python World"

print("Original String:", text)

# 1. String Slicing
print("Slice from index 0 to 5:", text[0:5])
print("Slice from index 6 to end:", text[6:])
print("Reverse String:", text[::-1])

# 2. Type Casting (String to other data types)
num_str = "123"
num_int = int(num_str)
num_float = float(num_str)

print("String to Integer:", num_int)
print("String to Float:", num_float)

# 3. String Functions

# (i) upper()
print("Uppercase:", text.upper())

# (ii) lower()
print("Lowercase:", text.lower())

# (iii) replace()
print("Replace Python with Java:", text.replace("Python", "Java"))

# (iv) find()
print("Position of 'World':", text.find("World"))

# (v) split()
print("Split String:", text.split())

#title case
print("convert into title",text.title())
```

Original String: Hello Python World
Slice from index 0 to 5: Hello
Slice from index 6 to end: Python World
Reverse String: dlroW nohtyP olleH

String to Integer: 123
String to Float: 123.0
Uppercase: HELLO PYTHON WORLD
Lowercase: hello python world
Replace Python with Java: Hello Java World
Position of 'World': 13
Split String: ['Hello', 'Python', 'World']

2. Implement a program based on Types of operator (Arithmetic Operator & Assignment Operator)

```
# Runtime input
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("\n--- Arithmetic Operators ---")
print("Addition (a + b):", a + b)
print("Subtraction (a - b):", a - b)
print("Multiplication (a * b):", a * b)
print("Division (a / b):", a / b)
print("Modulus (a % b):", a % b)
print("Floor Division (a // b):", a // b)
print("Exponentiation (a ** b):", a ** b)

print("\n--- Assignment Operators ---")

c = a
print("Initial value of c:", c)

c += b
print("c += b :", c)

c -= b
print("c -= b :", c)

c *= b
print("c *= b :", c)

c /= b
print("c /= b :", c)

c %= b
print("c %= b :", c)
```

3. Implement a program based on decision and looping statements.

```
n = int(input("Enter a number: "))

# Decision statement

if n % 2 == 0:
    print(n, "is Even")
else:
    print(n, "is Odd")

# Simple FOR loop (print even numbers)

print("\nEven numbers from 1 to", n, ":")
for i in range(2, n + 1, 2):
    print(i, end=" ")

# Simple WHILE loop (print sum of numbers)

sum = 0
i = 1
while i <= n:
    sum += i
    i += 1

print("\n\nSum of numbers from 1 to", n, "is:", sum)
```

output -

Enter a number: 5
Positive number

For loop output:
0 1 2 3 4

While loop output:
1 2 3 4 5

4. Implement a program based on using Tuple with its 10 functions.

```
# Creating a tuple with runtime input
t = tuple(input("Enter elements separated by space: ").split())

print("\nTuple:", t)

# 1. len()
print("1. Length of tuple:", len(t))

# 2. max()
print("2. Maximum element:", max(t))

# 3. min()
print("3. Minimum element:", min(t))

# 4. count()
item = input("Enter element to count: ")
print("4. Count of element:", t.count(item))

# 5. index()
item2 = input("Enter element to find index: ")
print("5. Index of element:", t.index(item2))

# 6. tuple slicing
print("6. Slicing (first 3 elements):", t[0:3])

# 7. concatenation
t2 = ('A', 'B')
print("7. Concatenation:", t + t2)

# 8. repetition
print("8. Repetition:", t * 2)

# 9. membership operator
print("9. Is 'A' in tuple?:", 'A' in t)

# 10. converting tuple to list
print("10. Tuple to list:", list(t))
```

output -

Enter elements separated by space: 10 20 30 40

Tuple: ('10', '20', '30', '40')

1. Length of tuple: 4

2. Maximum element: 40

3. Minimum element: 10

```
Enter element to count: 20
4. Count of element: 1
Enter element to find index: 30
5. Index of element: 2
6. Slicing (first 3 elements): ('10', '20', '30')
7. Concatenation: ('10', '20', '30', '40', 'A', 'B')
8. Repetition: ('10', '20', '30', '40', '10', '20', '30', '40')
9. Is 'A' in tuple?: False
10. Tuple to list: ['10', '20', '30', '40']
```

5. Implement a program based on Functions, Function with parameters.

1. Simple Function (No parameters)

```
def greet():
    print("Hello! Welcome to Python.")
```

Call the function

```
greet()
```

2. Function with Parameters

```
def add_numbers(a, b):
    sum = a + b
    print("Sum of", a, "and", b, "is:", sum)
```

Taking input from user

```
num1 = int(input("\nEnter first number: "))
num2 = int(input("Enter second number: "))
```

Call the function with parameters

```
add_numbers(num1, num2)
```

6. Implement a program based on using List with its 10 functions.

```
# 1. Creating a list with runtime input
lst = input("Enter list elements separated by space: ").split()
print("\nOriginal List:", lst)

# 1. len() - Length of list
print("1. Length of list:", len(lst))

# 2. max() - Maximum element
print("2. Maximum element:", max(lst))

# 3. min() - Minimum element
print("3. Minimum element:", min(lst))

# 4. append() - Add element at the end
lst.append("new_item")
print("4. After append():", lst)

# 5. insert() - Insert element at specific index
lst.insert(1, "inserted")
print("5. After insert():", lst)

# 6. remove() - Remove a specific element
if "new_item" in lst:
    lst.remove("new_item")
print("6. After remove():", lst)

# 7. pop() - Remove last element (or at index)
popped_item = lst.pop()
print("7. After pop():", lst, "| Popped item:", popped_item)

# 8. count() - Count occurrences of an element
count_item = input("Enter element to count: ")
print("8. Count of element:", lst.count(count_item))

# 9. sort() - Sort the list (only works if elements are comparable)
try:
    lst.sort()
    print("9. Sorted list:", lst)
except:
    print("9. Sort not possible for mixed types")

# 10. reverse() - Reverse the list
lst.reverse()
print("10. Reversed list:", lst)
```

output –

Enter list elements separated by space: 10 5 20 5

Original List: ['10', '5', '20', '5']

1. Length of list: 4

2. Maximum element: 5

3. Minimum element: 10

4. After append(): ['10', '5', '20', '5', 'new_item']

5. After insert(): ['10', 'inserted', '5', '20', '5', 'new_item']

6. After remove(): ['10', 'inserted', '5', '20', '5']

7. After pop(): ['10', 'inserted', '5', '20'] | Popped item: 5

Enter element to count: 5

8. Count of element: 1

9. Sort not possible for mixed types

10. Reversed list: [20, 5, 'inserted', '10']

7. Implement a program based on using Dictionaries with its (any 5) functions.

Simple program using dictionary functions

Creating a dictionary

```
student = {"101": "Rahul", "102": "Priya", "103": "Amit"}
```

1. len() function

```
print("Number of students:", len(student))
```

2. keys() function

```
print("Keys:", student.keys())
```

3. values() function

```
print("Values:", student.values())
```

4. get() function

```
print("Name of roll 102:", student.get("102"))
```

5. pop() function

```
student.pop("103")
```

```
print("After deleting one element:", student)
```

Number of students: 3

Keys: dict_keys(['101', '102', '103'])

Values: dict_values(['Rahul', 'Priya', 'Amit'])

Name of roll 102: Priya

After deleting one element: {'101': 'Rahul', '102': 'Priya'}

8. Implement a program based on Writing Text Files, Appending Text to a File, Reading Text Files.

```
f=open("C:/Users/parde/Desktop/my/myfile.txt","w")
f.write("\n file handling program")
f.close()
```

```
f=open("C:/Users/parde/Desktop/my/myfile.txt","a")
f.write("\n append file")
f.close()
```

```
f=open("C:/Users/parde/Desktop/my/myfile.txt","r")
content=f.read()
print("\n content")
print(content)
f.close()
```

9. Implement a program to Creating Layouts, Packing Order, Controlling Widget Appearances.

```
from tkinter import *

# Create main window
root = Tk()
root.title("Tkinter Layout Example")
root.geometry("300x250")

# Creating widgets with appearance control
label1 = Label(root, text="Top Label", bg="lightblue", fg="black", font=("Arial", 12))
label2 = Label(root, text="Bottom Label", bg="lightgreen", fg="black", font=("Arial", 12))

button1 = Button(root, text="Left Button", bg="orange", fg="white", width=12)
button2 = Button(root, text="Right Button", bg="purple", fg="white", width=12)

# Packing order and layout control
label1.pack(side=TOP, fill=X, padx=5, pady=5)
label2.pack(side=BOTTOM, fill=X, padx=5, pady=5)

button1.pack(side=LEFT, padx=10, pady=20)
button2.pack(side=RIGHT, padx=10, pady=20)

# Run the application
root.mainloop()
```


10. Implement a program to demonstrate Class and Object.

Program to demonstrate Class and Object

Defining a class

class Student:

Method to get data

def get_data(self, name, roll):

self.name = name

self.roll = roll

Method to display data

def display(self):

print("Student Name:", self.name)

print("Roll Number:", self.roll)

Creating object of the class

s1 = Student()

Calling methods using object

s1.get_data("Rahul", 101)

s1.display()

11. Implement a program based on built-in functions (any 10) .

Program to demonstrate 10 Python built-in functions

1. abs() - absolute value

num = int(input("Enter a number for abs(): "))

print("Absolute value:", abs(num))

2. len() - length of a string or list

text = input("Enter a string for len(): ")

print("Length of the string:", len(text))

3. max() - maximum value

numbers = list(map(int, input("Enter numbers separated by space for max(): ").split()))

print("Maximum number:", max(numbers))

4. min() - minimum value

print("Minimum number:", min(numbers))

```

# 5. sum() - sum of numbers
print("Sum of numbers:", sum(numbers))

# 6. round() - round a floating point number
flt = float(input("Enter a float number for round(): "))
print("Rounded value:", round(flt))

# 7. sorted() - sort a list
print("Sorted numbers:", sorted(numbers))

# 8. str() - convert to string
print("Convert number to string:", str(num))

# 9. type() - check data type
print("Data type of variable 'num':", type(num))

# 10. input() - read input from user (already used above)
name = input("Enter your name using input(): ")
print("Hello,", name)

```

12. Implement a program to demonstrate Array. (any 5 functions)

Program to demonstrate array (list) functions

Creating an array

```
arr = [10, 20, 30, 40, 50]
```

```
print("Original Array:", arr)
```

1. append() – add element at end

```
arr.append(60)
```

```
print("After append:", arr)
```

2. insert() – insert element at specific position

```
arr.insert(2, 25)
```

```
print("After insert:", arr)
```

3. remove() – remove specific element

```
arr.remove(40)
```

```
print("After remove:", arr)
```

4. pop() – remove last element

```
arr.pop()
print("After pop:", arr)

# 5. sort() – sort the array
arr.sort()
print("After sort:", arr)
```

1 . Implement a program based on Writing Text Files, Appending Text to a File, Reading Text Files.

2 . Implement a program based on paths and directories, file information, naming, moving, copying, and removing files.

```
import os
import shutil

# Create a directory
os.mkdir("MyFolder")
print("Directory created")

# Create a file inside the directory
file_path = "MyFolder/sample.txt"
file = open(file_path, "w")
file.write("This is a sample file.")
file.close()
print("File created")

# File information
print("File exists:", os.path.exists(file_path))
print("File size:", os.path.getsize(file_path), "bytes")

# Rename the file
new_name = "MyFolder/data.txt"
os.rename(file_path, new_name)
print("File renamed")

# Copy the file
shutil.copy(new_name, "MyFolder/data_copy.txt")
print("File copied")

# Move the copied file
shutil.move("MyFolder/data_copy.txt", "data_copy.txt")
print("File moved")

# Remove the moved file
os.remove("data_copy.txt")
print("File removed")
```

```
# Remove directory
os.remove(new_name)
os.rmdir("MyFolder")
print("Directory removed")
```

output –
Directory created
File created
File exists: True
File size: 22 bytes
File renamed
File copied
File moved
File removed
Directory removed

3 . Implement a program based on Functions, Function with parameters.

4. Implement a program based on Types of operator (Arithmetic Operator , Logical Operator)

Program to demonstrate Arithmetic and Logical Operators

```
# Taking two numbers as input
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
```

```
# Arithmetic Operators
print("\nArithmetic Operators:")
print("Addition (a + b):", a + b)
print("Subtraction (a - b):", a - b)
print("Multiplication (a * b):", a * b)
print("Division (a / b):", a / b)
print("Modulus (a % b):", a % b)
print("Exponent (a ** b):", a ** b)
print("Floor Division (a // b):", a // b)
```

```
# Logical Operators
print("\nLogical Operators:")
print("a > 10 and b > 5:", a > 10 and b > 5)
print("a > 10 or b > 5:", a > 10 or b > 5)
print("not(a > 10):", not(a > 10))
```

output –
Enter first number: 12
Enter second number: 6

Arithmetic Operators:

Addition (a + b): 18
Subtraction (a - b): 6
Multiplication (a * b): 72
Division (a / b): 2.0
Modulus (a % b): 0
Exponent (a ** b): 2985984
Floor Division (a // b): 2

Logical Operators:

a > 10 and b > 5: True

a > 10 or b > 5: True

not(a > 10): False

4. Implement a program based on using Tuple with its 10 functions.

5. Implement a program creating Types of GUI Widgets, Resizing, and Configuring Options.

```
from tkinter import *

# Create main window
root = Tk()
root.title("GUI Widgets Example")
root.geometry("400x300") # Resize window: width x height
root.resizable(True, True) # Allow resizing (width, height)

# Label widget
label = Label(root, text="Welcome to Tkinter GUI", bg="lightblue", fg="black", font=("Arial", 14))
label.pack(pady=10)

# Entry widget
Label(root, text="Enter your name:").pack()
entry = Entry(root, width=25, font=("Arial", 12))
entry.pack(pady=5)

# Button widget
def greet():
    name = entry.get()
    message.config(text=f"Hello, {name}!")

button = Button(root, text="Greet", bg="green", fg="white", command=greet)
button.pack(pady=10)

# Checkbutton widget
chk_var = IntVar()
check = Checkbutton(root, text="Subscribe to newsletter", variable=chk_var)
check.pack(pady=5)
```

```

# Radiobutton widget
gender = StringVar()
Label(root, text="Select Gender:").pack()
Radiobutton(root, text="Male", variable=gender, value="Male").pack()
Radiobutton(root, text="Female", variable=gender, value="Female").pack()

# Message label to show greeting
message = Label(root, text="", font=("Arial", 12), fg="blue")
message.pack(pady=10)

# Run the GUI
root.mainloop()

```

6. same as up.

10. Implement a program to create, import and use package and modules.

11 . Implement a program to Create Student Information using types of GUI widgets.
from tkinter import *

```

# Create main window
root = Tk()
root.title("Student Information")
root.geometry("350x350")

# Variables
gender = StringVar()
course_python = IntVar()
course_java = IntVar()

# Labels and Entry widgets
Label(root, text="Student Information Form", font=("Arial", 14)).pack(pady=10)

Label(root, text="Student Name").pack()
Entry(root).pack()

Label(root, text="Roll Number").pack()
Entry(root).pack()

# Radio buttons (Gender)

```

```

Label(root, text="Gender").pack()
Radiobutton(root, text="Male", variable=gender, value="Male").pack()
Radiobutton(root, text="Female", variable=gender, value="Female").pack()

# Check buttons (Courses)
Label(root, text="Courses").pack()
Checkbutton(root, text="Python", variable=course_python).pack()
Checkbutton(root, text="Java", variable=course_java).pack()

# Submit button
Button(root, text="Submit", bg="green", fg="white").pack(pady=20)

# Run the application
root.mainloop()

```

12 Implement a program based on paths and directories, file information, naming, moving, copying, and removing files.

```

os.mkdir(new_dir)
print(f"Directory created: {new_dir}")
# Step 2: File Information
file_name = os.path.join(new_dir, "sample.txt")
with open(file_name, 'w') as f:
    f.write("This is a sample file.")
print("\nFile Information")
print(f"File Path: {file_name}")
print(f"File Exists: {os.path.exists(file_name)}")
print(f"File Size: {os.path.getsize(file_name)} bytes")
print(f"Last Modified: {os.path.getmtime(file_name)}")
print(f"Absolute Path: {os.path.abspath(file_name)}")
# Step 3: Renaming the file
new_file_name = os.path.join(new_dir, "renamed_sample.txt")
os.rename(file_name, new_file_name)
print(f"\nFile Renamed to: {new_file_name}")
# Step 4: Moving the file
moved_file_path = os.path.join(current_dir, "renamed_sample.txt")
shutil.move(new_file_name, moved_file_path)
print(f"File Moved to: {moved_file_path}")
# Step 5: Copying the file
copied_file_path = os.path.join(current_dir, "copied_sample.txt")
shutil.copy(moved_file_path, copied_file_path)
print(f"File Copied to: {copied_file_path}")

# Step 6: Removing the files
os.remove(moved_file_path)
print(f"File Removed: {moved_file_path}")
os.remove(copied_file_path)

```

```
print(f"Copied File Removed: {copied_file_path}")  
# Clean up: Remove the directory  
os.rmdir(new_dir)  
print(f"Directory Removed: {new_dir}")
```